

基于 CUDA 小矩阵加速的机械臂批量逆运动学求解

刘小明^{1,*}, 某某某²

(1. XXXX 大学 XXXX 学院, XX XX XXXXX; 2. XXXX 研究所, XX XXXXX)

文章编号: (预留) DOI: (预留)

摘要: 针对机械臂批量逆运动学求解中 6×6 小矩阵反复求解和多迭代带来的 kernel launch 累积开销问题, 提出一种基于计算统一设备架构 (compute unified device architecture, CUDA) 底层优化的批量阻尼最小二乘法加速方法。该方法将每个目标映射为 1 个 CUDA 线程块 (128 线程), 在单次 kernel launch 中完成全部迭代; 设计寄存器驻留的 6×6 LDL^T 求解器替代 cuBLAS 库调用; 采用列填充共享内存布局降低 Bank 冲突; 引入 FP32/FP64 混合精度策略——正运动学和雅可比计算使用 FP32, LDL^T 求解与收敛判定保留 FP64。在 NVIDIA GeForce RTX 4060 Laptop GPU 上, 以官方 UR10 为实验对象构建统一 benchmark。实验表明: 混合精度配置在批量规模 $N = 100$ 时吞吐达 112,414 targets/s, 为 NVIDIA cuRobo 的 36.1 倍; 消融实验中自适应阻尼将收敛率从 52–83% 恢复至 100%, 混合精度在此基础上额外提升吞吐 120–149% 且收敛率无退化; $N = 100 \rightarrow 10,000$ 全量程上 GPU 时间严格线性 ($R^2 > 0.999$), 吞吐稳定在 148k–174k targets/s ($\pm 8\%$)。Nsight Compute 剖析确认 kernel 为计算密集型, Bank 冲突降低 63%, 零寄存器溢出。

关键词: 逆运动学; 计算统一设备架构; 小矩阵求解; 混合精度; Bank 冲突

中图分类号: TP391.4 **文献标志码:** A

收稿日期: (预留); **修回日期:** (预留); **网络优先出版日期:** (预留)。

引用格式: 刘小明, 某某某. 基于 CUDA 小矩阵加速的机械臂批量逆运动学求解 [J]. 系统工程与电子技术, 2026, XX(X): XXX–XXX.

CUDA-Accelerated Small-Matrix Solver for Batch Inverse Kinematics of Robot Manipulators

LIU Xiaoming^{1,*}, XXXXX²

(1. School of XXXX, XXXX University, XX XXXXX, China; 2. XXXX Research Institute, XX XXXXX, China)

Abstract: Batch inverse kinematics (IK) for robot manipulators incurs substantial kernel launch overhead from repeatedly solving 6×6 linear systems across numerous targets. This paper presents a compute unified device architecture (CUDA)-based low-level acceleration method for batch damped least-squares IK. Each target is mapped to one CUDA block (128 threads), with all iterations encapsulated within a single kernel launch. A register-resident 6×6 LDL^T solver replaces cuBLAS library calls in 86 scalar operations. A column-padded shared-memory layout reduces bank conflicts by 63%. A mixed-precision strategy applies FP32 to forward kinematics and Jacobian evaluation while retaining FP64 for LDL^T factorization and convergence checks. Experiments on an official UR10 model with unified targets and thresholds on an NVIDIA GeForce RTX 4060 Laptop GPU demonstrate that the mixed-precision configuration achieves 112,414 targets/s at batch size $N = 100$ ($36.1 \times$ NVIDIA

cuRobo). Ablation shows adaptive damping restores convergence rate from 52–83% to 100%, and mixed precision adds 120–149% throughput gain without convergence degradation. GPU time scales strictly linearly across $N = 100$ –10,000 ($R^2 > 0.999$) with throughput of 148k–174k targets/s ($\pm 8\%$). Nsight Compute profiling confirms the kernel is compute-bound with zero register spilling.

Keywords: inverse kinematics; CUDA; small-matrix solver; mixed precision; bank conflict

1 引言

机械臂批量逆运动学 (inverse kinematics, IK) 求解是轨迹优化、料箱抓取和采样规划等机器人应用中的核心计算环节 [1]。给定 N 个末端执行器目标位姿, 批量 IK 需为每个目标求解一组关节角, 使正运动学 (forward kinematics, FK) 输出与目标位姿匹配。六自由度串联机械臂的 IK 通常无闭式解, 实际系统普遍采用阻尼最小二乘法 (damped least squares, DLS) 或 Levenberg-Marquardt (LM) 法进行数值迭代 [2–3]。贾龙飞等 [4] 系统综述了冗余机械臂 IK 的三大类求解方法——解析法、数值解法和智能算法——指出数值解法通用性好但计算量大, 实时性难以兼顾。批量 IK 的计算特征可归纳为三重挑战: (1) 每次迭代需反复求解 6×6 线性系统, 矩阵规模固定且极小; (2) 每个目标

需 10–80 次迭代才能收敛；(3) 独立目标数 N 可达数千至上万。

批量 IK 计算方法。 Orocos KDL 提供了成熟的 C++ FK/IK 工具链 [5]，但其单线程串行迭代模式在批量场景下吞吐有限（约 986 targets/s）。Buss 等 [2] 系统比较了 Jacobian 转置、伪逆和 DLS 三种方法的收敛特性，指出 DLS 在奇异点附近的数值稳定性优于纯伪逆方法。贾龙飞等 [4] 的综述进一步指出，现有 IK 加速研究集中在算法层面（改进粒子群、人工蜂群等群体智能方法），尚未涉及 GPU 硬件加速。

GPU 加速逆运动学。 NVIDIA cuRobo[6] 将 IK 建模为非线性优化问题，在 GPU 上并行评估数千粒子，通过 L-BFGS 求解，在 RTX 4090 上实现标准 IK 37,000 queries/s。HJCD-IK[7] 结合贪婪坐标下降初始化和 Jacobian 伪逆精化。ManipulaPy[8] 基于指数积公式和 CuPy 自定义 CUDA kernel，在轨迹生成上实现 40 倍 CPU 加速。Kino-PAX[9] 和 SPaSM[10] 等 GPU 运动规划工作在算法层面实现了高度并行化，其“全 GPU 原生”的架构理念启发了本文的单 kernel 设计思路。然而上述方法均基于 PyTorch、JAX 或 CuPy 等高层框架。

CUDA 底层优化。 面向极小矩阵 ($n \leq 32$) 的 CUDA 底层优化在数值线性代数领域已有深入研究。Abdelfattah 等 [11] 针对 ≤ 32 矩阵、百万级批量的 LU/QR/Cholesky 分解，通过寄存器分块实现相对厂商库 11.8 倍加速。刘世芳等 [12] 在 GPU 上实现了批量 LU 分解的矩阵求逆，在 TITAN V 上达到 cuBLAS 的 13 倍加速。Shridhar[13] 在 RTX 3080 上证实自定义小矩阵求逆 kernel 仅占总运行时间的 10%，而 cuBLAS 调用占 67%。在混合精度方面，Abdelfattah 等 [14] 利用 Tensor Core 进行三层迭代精化，在关键路径保留 FP64 的前提下实现 3–5 倍加速。

本文的核心观点是：对 6×6 这一固定规模的 IK 小矩阵，**手写 CUDA 底层实现可以系统性地优于基于通用库和框架的方案**。具体贡献为：(1) 1 block/target 映射与单 kernel 全迭代封装，将 kernel launch 开销从 $O(N \cdot K)$ 降至 $O(1)$ ；(2) 寄存器驻留的 6×6 LDL^T 求解器，86 次标量运算替代 cuBLAS 库调用；(3) PaddedMat 6×8 共享内存布局将 Bank 冲突降低 63%；(4) FP32/FP64 混合精度策略，在 Ada Lovelace 64:1 的吞吐比下释放显著性能增益且收敛率无退化。

2 批量 IK 问题建模与 DLS 方法

2.1 运动学模型

UR10 为 6 自由度全旋转关节串联机械臂，其 DH 参数遵循 Universal Robots 官方统一机器人描述格式 (unified robot description format, URDF) [15]。正运动学将关节角向量 $\mathbf{q} = [q_1, \dots, q_6]^T \in \mathbb{R}^6$ 映射为末端执行器齐次变换矩阵 $\mathbf{T}_{ee}(\mathbf{q}) \in \text{SE}(3)$ ：

$$\mathbf{T}_{ee}(\mathbf{q}) = \prod_{i=1}^6 \mathbf{T}_{i-1,i}(q_i) \quad (1)$$

其中 $\mathbf{T}_{i-1,i}(q_i) = \exp(\hat{\xi}_i \cdot q_i) \cdot \mathbf{T}_{i-1,i}(0)$ 。实用中采用 Rodrigues 公式： $\mathbf{R}_i = \mathbf{I} + \sin(q_i)[\mathbf{a}_i]_{\times} + (1 - \cos(q_i))[\mathbf{a}_i]_{\times}^2$ ，其中 $[\mathbf{a}_i]_{\times}$ 为关节轴 $\mathbf{a}_i$ 的反对称矩阵。

对目标位姿 $\mathbf{T}_{tgt} \in \text{SE}(3)$ ，定义位姿误差 $\mathbf{e}(\mathbf{q}) \in \mathbb{R}^6$ ：

$$\mathbf{e}_p(\mathbf{q}) = \mathbf{p}(\mathbf{T}_{ee}) - \mathbf{p}(\mathbf{T}_{tgt}) \quad (2)$$

$$\mathbf{e}_R(\mathbf{q}) = \log(\mathbf{R}_{ee} \cdot \mathbf{R}_{tgt}^{\top})^{\vee} \quad (3)$$

其中 $\mathbf{p}(\cdot)$ 提取平移分量， $\log(\cdot)^{\vee}$ 为 $\text{SO}(3)$ 上的对数映射。引入对角权重矩阵 $\mathbf{W} = \text{diag}(w_p, w_p, w_p, w_R, w_R, w_R)$ 。

2.2 DLS 迭代与批量特征

DLS[2-3] 在第 k 次迭代中极小化正则化目标：

$$\min_{\Delta \mathbf{q}} \frac{1}{2} \|\mathbf{W} \cdot \mathbf{J}(\mathbf{q}^{(k)}) \cdot \Delta \mathbf{q} + \mathbf{W} \cdot \mathbf{e}(\mathbf{q}^{(k)})\|^2 + \frac{1}{2} \lambda \|\Delta \mathbf{q}\|^2 \quad (4)$$

其中 $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times 6}$ 为雅可比矩阵。区别于以误差 $\mathbf{e}$ 为变量的定义 ($\mathbf{J} = \partial \mathbf{e} / \partial \mathbf{q}$)，本文采用基于 FK 输出的定义—— $\mathbf{J} = \partial \mathbf{x} / \partial \mathbf{q}$ ， $\mathbf{x}(\mathbf{q}) = [\mathbf{p}(\mathbf{T}_{ee}); \log(\mathbf{R}_{ee})^{\vee}]$ 。

雅可比由 FK 中心差分计算：

$$\mathbf{J}_{:,j} = \frac{\mathbf{x}(\mathbf{q} + \varepsilon \mathbf{e}_j) - \mathbf{x}(\mathbf{q} - \varepsilon \mathbf{e}_j)}{2\varepsilon}, \quad \varepsilon = 10^{-6} \text{ rad} \quad (5)$$

每列需 2 次 FK 调用，每次 DLS 迭代合计 12 次 FK。令式 (4) 梯度为零，得阻尼正规方程：

$$(\mathbf{J}^{\top} \mathbf{W}^2 \mathbf{J} + \lambda \mathbf{I}) \cdot \Delta \mathbf{q} = -\mathbf{J}^{\top} \mathbf{W}^2 \mathbf{e} \quad (6)$$

记 $\mathbf{H} = \mathbf{J}^{\top} \mathbf{W}^2 \mathbf{J} + \lambda \mathbf{I} \in \mathbb{R}^{6 \times 6}$ 为阻尼 Hessian 矩阵， $\mathbf{g} = \mathbf{J}^{\top} \mathbf{W}^2 \mathbf{e} \in \mathbb{R}^6$ 为梯度，则 $\Delta \mathbf{q} = -\mathbf{H}^{-1} \mathbf{g}$ 。更新 $\mathbf{q}^{(k+1)} = \mathbf{q}^{(k)} + \Delta \mathbf{q}$ 并投影至 $[\mathbf{q}_{\min}, \mathbf{q}_{\max}]$ 。收敛条件： $\|\mathbf{e}_p\| < 10 \text{ mm}$ 且 $\|\mathbf{e}_R\| < 5^\circ$ (0.0873 rad)， $K_{\max} = 160$ 。

批量计算特征分析。 N 个目标共享相同模型参数但各自拥有独立状态向量 $(\mathbf{q}, \mathbf{e}, \mathbf{J}, \mathbf{H}, \mathbf{g}, \Delta \mathbf{q})$ 。这一结构具有两个关键性质：(1) **完全数据并行**——各目标的

DLS 迭代无任何依赖关系；(2) 计算粒度极小——单目标每次迭代的浮点运算量约 1,500 FLOP (12 次 FK + 1 次 LDL^T + 阻尼更新)，远小于 GPU 上典型 kernel 的计算规模。

3 CUDA 小矩阵加速设计

3.1 1 Block/Target 并行映射与单 Kernel 封装

Grid-Block 映射设计。本文采用 $Grid(N, 1, 1) + Block(128, 1, 1)$ 的二维映射。选择 128 线程/block 而非 256 的理由是：kernel 的寄存器用量已达 94–98/线程，128 线程下每 block 寄存器总量为 12,544，每个流式多处理器 (streaming multiprocessor, SM) 可同时驻留约 5 个 block； 6×6 矩阵规模下可并行的最大线程数由 $\mathbf{H}$ 矩阵的 36 个独立元素决定，128 线程已提供充足并行粒度。

阶段式 flat threadIdx.x 分工。表 1 给出了每次 DLS 迭代中各计算阶段的线程分配。

表 1: DLS 迭代各阶段线程分工

计算阶段	线程范围	并行度
FK/位姿误差/收敛判定	$t = 0$	1
数值 Jacobian	$0 \leq t < 6$	6
自适应阻尼更新	$t = 0$	1
$\mathbf{H}$ 矩阵构造	$0 \leq t < 36$	36
$\mathbf{g}$ 向量构造	$0 \leq t < 6$	6
LDL^T 求解/步长钳位	$t = 0$	1
关节更新 + 限位投影	$0 \leq t < 6$	6

注： $\mathbf{H}$ 矩阵构造使用线程 0–35，跨越 Warp 0 (lane 0–31) 和 Warp 1 (lane 0–3)，不可将 $\mathbf{H}$ 矩阵计算归属于任一单独 warp。

算法 1 DLS 单次迭代 (per-block, 128 线程)

Listing 1: DLS 单次迭代伪代码

```

1 // Input: s_q[6] (shared mem, FP64), s_T_tgt
  //         [16], s_lambda
2 // Output: s_q[6] (updated joint angles)
3
4 if (threadIdx.x == 0) { // FK
5     computation
6     forward_kinematics(s_q, s_T_ee);
7     pose_error(s_T_ee, s_T_tgt, s_err);
8 }
9 __syncthreads();

```

```

9 if (threadIdx.x == 0) //
10     convergence check
11     converged = (norm(e_p) < 0.01) && (norm(e_R)
12         < 0.0873);
13 __syncthreads();
14 if (threadIdx.x < 6) //
15     numerical Jacobian, 6 columns
16     J_col[tid] = (FK(q+eps) - FK(q-eps)) / (2*eps);
17 __syncthreads();
18 if (threadIdx.x == 0) //
19     adaptive damping
20     update_lambda(s_lambda, pos_err);
21 __syncthreads();
22 if (threadIdx.x < 36) { // H = J
23     ~T W^2 J + lambda*I
24     int row = threadIdx.x / 6;
25     int col = threadIdx.x % 6;
26     double sum = 0.0;
27     for (int k = 0; k < 6; k++)
28         sum += J(k, row) * w2[k] * J(k, col);
29     if (row == col) sum += s_lambda; //
30     damping on diagonal
31     H(row, col) = sum;
32 }
33 __syncthreads();
34 if (threadIdx.x < 6) { // g = J
35     ~T W^2 e
36     double sum = 0.0;
37     for (int k = 0; k < 6; k++)
38         sum += J(k, tid) * w2[k] * e[k];
39     g[tid] = sum;
40 }
41 __syncthreads();
42 if (threadIdx.x == 0) { // LDLT
43     solve + step clamp
44     ldlt_solve_6x6(H, g, dq);
45     double nrm = norm_inf(dq);
46     if (nrm > 0.35) // 0.35
47         rad clamp
48         for (int i = 0; i < 6; i++) dq[i] *=
49             0.35/nrm;
50 }
51 __syncthreads();
52 if (threadIdx.x < 6) // joint
53     update + limits
54     s_q[tid] = clamp(s_q[tid]+dq[tid], q_min[tid]
55         , q_max[tid]);

```

单 Kernel 全迭代封装——与 cuRobo 的结构性对比。全部 $K_{\max} = 160$ 次 DLS 迭代在单次 `ik_batch_solve<<N, 128>>>` 启动中完成。CPU 端仅需一次 `cudaDeviceSynchronize()`，迭代过程中零 host-device 往返。该设计的直接动机来自对 cuRobo 的 Nsight Systems CUDA API trace 实测分析。对于 cuRobo，每批次的执行路径为：Python 调用 → Py-

Torch tensor 分配 → cuRobo 内部子批次 (sub-batch) 划分 → 每个子批次启动多个 CUDA kernel → 粒子评估与 L-BFGS 迭代 → 结果回传。在 $N = 4000$ (退化点), 该流程产生了 14,108 次 CUDA kernel launch、5,069 次 cudaEventRecord 和 4,985 次 cudaStreamWaitEvent 调用, 每次 launch 在消费级 GPU 驱动栈上耗时约 5–10 μs , 仅启动开销即累积至 70–140 ms。相比之下, 本文方案在任意 N 值下 kernel launch 数恒为 1——所有 N 个 block 在同一 launch 中由 GPU 调度器无锁并行发射, 完全消除了跨 kernel 同步和多次 launch 的累积开销。

与 UR10 运动学的联动设计。 本文的并行映射利用了 UR10 批量 IK 的特殊结构: N 个目标共享同一套 DH 参数和关节限位 (通过常量内存广播到所有 block), 但各自维护独立的状态向量。这一“同模型、异状态”的特性使每个 block 成为一个自包含的 IK 求解单元——无需 block 间通信, 无需动态内存分配, 无需 GPU-CPU 同步。在实际实现中, 每个目标约需 2 KB 中间状态 (6 个关节角 $\times$ 8 字节 + 6×6 Jacobian $\times$ 8 字节 + 6×6 Hessian $\times$ 8 字节 + 误差/梯度/步长缓冲区), $N = 10,000$ 时总全局内存需求约 20 MB, 远低于 8 GB 显存上限。因此批量扩展仅受 GPU 全局内存容量约束, 预计 $N = 10^5$ 时 (约 200 MB) 仍可维持线性扩展。

3.2 寄存器级 6×6 LDL^T 求解器

不使用 cuBLAS 的理由。 cuBLAS 的批量接口针对 $n \geq 32$ 的矩阵优化, 调用需经历句柄创建 (5–10 μs) 和数据搬运。对 6×6 矩阵, 求解本身仅需约 0.1 μs , cuBLAS 开销是其 50–100 倍。文献 [13] 的实验验证了这一判断。

LDL^T 替代 Cholesky 的理由。 Cholesky ($\mathbf{H} = \mathbf{LL}^T$) 需 6 次 sqrt, GPU 上 sqrt 吞吐仅为 FMA 的 $1/4 \sim 1/8$ [16]。LDL^T ($\mathbf{H} = \mathbf{LDL}^T$) 避免所有 sqrt。 $\lambda > 0$ 保证 $\mathbf{H} \succ 0$, $d_j > 0$ 恒成立。

运算量与寄存器驻留。 LDL^T 四阶段: 分解 (35 FMA + 15 DIV)、前代 $\mathbf{Ly} = \mathbf{g}$ (15 FMA)、对角缩放 $\mathbf{z} = \mathbf{D}^{-1}\mathbf{y}$ (6 DIV)、回代 $\mathbf{L}^T\mathbf{x} = \mathbf{z}$ (15 FMA), 合计 86 次标量运算, 全部驻留寄存器。

算法 2 寄存器级 6×6 LDL^T 求解器

Listing 2: LDLT 求解器 (86 次标量运算)

```
1 // Phase 1: LDLT decomposition (35 FMA + 15 DIV)
2 for (int j = 0; j < 6; j++) {
3     double d = H[j][j];
4     for (int k = 0; k < j; k++) //
        diagonal update
```

```
5     d -= L[j][k] * L[j][k] * D[k];
6     D[j] = d;
7     for (int i = j+1; i < 6; i++) {
8         double sum = H[i][j];
9         for (int k = 0; k < j; k++) // off-
            diagonal update
10            sum -= L[i][k] * L[j][k] * D[k];
11        L[i][j] = sum / D[j]; // scale
            by D[j]
12    }
13    L[j][j] = 1.0;
14 }
15 // Phase 2: forward substitution Ly = g (15 FMA)
16 for (int i = 0; i < 6; i++) {
17     double sum = g[i];
18     for (int k = 0; k < i; k++)
19        sum -= L[i][k] * y[k];
20    y[i] = sum;
21 }
22 // Phase 3: diagonal scaling z = D^{-1} y (6 DIV)
23 for (int i = 0; i < 6; i++)
24    z[i] = y[i] / D[i];
25 // Phase 4: backward substitution L^T x = z (15
    FMA)
26 for (int i = 5; i >= 0; i--) {
27     double sum = z[i];
28     for (int k = i+1; k < 6; k++)
29        sum -= L[k][i] * x[k];
30    x[i] = sum;
31 }
```

$n = 6$ 为编译时常量, NVCC #pragma unroll 将三重循环 ($j=0..5, k=0..j-1, i=j+1..5$) 全部展开为直线代码, 消除分支预测失败和循环控制指令开销。Nsight Compute 实测: FP64 全优化 94 regs/thread, CUDA 混合精度 98 regs/thread (+4 来自 FP32 $\leftrightarrow$ FP64 转换中间变量), 均低于 Ada Lovelace 上限 255, 零 local memory spill——所有中间变量 ($\mathbf{L}[6][6]$ 、 $\mathbf{D}[6]$ 、 $\mathbf{y}[6]$ 、 $\mathbf{z}[6]$) 完全容纳于寄存器文件。

与 cuBLAS 的调用链路对比。 cuBLAS 的 cublasDgetrfBatched 调用链路为: (1) 共享内存中 Jacobian 和 Hessian 通过全局内存拷贝到 host 端可见缓冲区 → (2) cuBLAS 句柄创建或复用 ($\sim 5 \mu\text{s}$) → (3) cuBLAS 内部分发到 batched kernel ($\sim 2 \sim 5 \mu\text{s}$) → (4) 结果通过全局内存拷贝回共享内存。该链路的总延迟约为 10–15 μs , 而本文的寄存器 LDL^T 求解仅需约 0.1 μs ——后者快 100–150 倍。文献 [13] 在第 3 代 RTX 3080 (Ampere, SM 8.6) 上独立验证了这一结论: 自定义小矩阵求逆 kernel 仅占总运行时间的 10%, cuBLAS 调用占 67%, 总体加速 4–6.5 倍。本文在 Ada Lovelace (SM 8.9) 上的结果与之一致。

3.3 PaddedMat 6×8 共享内存 Bank 冲突降低

Bank 冲突机制的硬件根源。 GPU 共享内存以 32 个 Bank 组织, 每 Bank 4 字节宽, 每时钟周期可同时服务最多 32 个来自不同 Bank 的访问请求。当同一 warp 内多个线程访问同一 Bank 的不同地址时, 请求被串行化处理, 导致有效带宽下降。FP64 元素 (8 字节) 天然跨 2 个连续 Bank (Bank $2c$ 和 Bank $2c+1$)。在自然 stride=6 行优先布局下, 行步长为 $6 \times 8 = 48$ 字节 = 12 Bank, Bank 访问模式由 $\gcd(12, 32) = 4$ 决定, 以 8 次为周期重复, 每周期内同一 Bank 被 2-3 个线程同时访问, 产生 2-3 路冲突。在 DLS 的 Jacobian 组装阶段 (6 个线程以列优先模式并发读取 Jacobian 各行) 和 H 矩阵构造阶段 (36 个线程以行优先 + 列优先混合模式并发访问), Bank 冲突导致约 10-20% 的有效共享内存带宽损失。

PaddedMat 6×8 设计。 行步长 $6 \rightarrow 8$, 步长变为 $8 \times 8 = 64$ 字节 = 16 Bank, $\gcd(16, 32) = 16$ 。关键性质: 偶数行 ($r = 0, 2, 4$) Bank 0-15, 奇数行 ($r = 1, 3, 5$) Bank 16-31, 两组完全不重叠。选择 stride=8 而非 16 的理由: stride=8 已实现 Bank 集完全分离, 更大 stride 不再带来额外收益, 且填充开销更小 (192 bytes/矩阵 vs 768 bytes/矩阵)。

Nsight Compute 实测。 $N = 100$ 下: FP64 全优化 3,522 $\rightarrow$ CUDA 混合精度 1,295 次 Bank 冲突 (-63%)。混合精度额外收益来自 FP32 数据宽度减半 (4 字节仅跨 1 Bank)。残余 1,295 次来自非 PaddedMat 6×8 路径, 优化效果表述为“降低”而非“消除”。

3.4 FP32/FP64 混合精度计算路径

硬件动机与设计空间。 Ada Lovelace 消费级 GPU 的 FP64 与 FP32 吞吐比为 1:64[16]。以 RTX 4060 Laptop 为例, 全 GPU 3,072 个 CUDA Core 中仅约 48 个等效 FP64 单元 (每 SM 约 2 个 FP64 单元), 传统全 FP64 DLS 将约 90% 的浮点运算提交给这 48 个单元, 导致大规模 FP32 算力闲置。混合精度设计需在三个候选方案中选择: (a) 全 FP32——吞吐最高, 但 $\text{LDL}^\top$ 求解中的除法运算 (21 次 DIV) 对 D_j 偏小极为敏感, FP32 的 7 位精度在条件数 $\kappa(\mathbf{H}) > 10^4$ 时可能导致步长方向严重偏离; (b) FP16+FP32——可利用 Tensor Core 加速矩阵乘法, 但 FP16 的 3.3 位十进制精度在 $\varepsilon = 10^{-6}$ 差分步长下, Jacobian 各列的相对舍入误差达 10^{-3} 量级, 不可接受; (c) FP32+FP64——FP32 提供 6-7 位十进制精度 ($\eta_{\text{FP32}} \approx 6 \times 10^{-8}$), FP64 为 $\text{LDL}^\top$ 分解中的 21 次除法和阻尼调节保留 $\approx 10^{-15}$ 的

相对精度裕度。本文选择方案 (c)。

FP32 $\leftrightarrow$ FP64 转换开销分析。 混合精度引入了类型转换指令。在 CUDA 中, $_d2f_rd()$ (FP64 $\rightarrow$ FP32, 向最近偶数舍入) 和 $_f2d_rn()$ (FP32 $\rightarrow$ FP64) 均为单周期指令, 吞吐与 FMA 相同。本文的转换点在 H 矩阵构造入口 (Jacobian FP32 值 $\rightarrow$ FP64 累加器) 和关节更新出口 (FP64 步长 $\rightarrow$ FP32 关节值), 每迭代仅 78 次转换 (36 个 H 元素 + 6 个 g 分量 + 36 个 Jacobian 元素), 占总指令数 $< 0.01\%$ 。NCU 实测确认类型转换未成为瓶颈。

表 2: 混合精度分配策略

计算阶段	精度	理由
FK 链式乘法	FP32	吞吐为 FP64 的 64 倍
Jacobian 各列	FP32	$\varepsilon = 10^{-6}$ 下舍入误差约 10^{-7}
H 矩阵累加	FP64	36 个内积抑制截断误差
g 向量累加	FP64	同上
$\text{LDL}^\top$ 求解	FP64	除法对 D_j 敏感
位姿误差/阻尼调节	FP64	精度关键

精度边界验证。 Jacobian 差分分子 $\approx 2 \times 10^{-6}$, FP32 机器精度 $\eta_{\text{FP32}} \approx 6 \times 10^{-8}$ 。关键保护机制: $\mathbf{J}^\top \mathbf{W}^2 \mathbf{J}$ 内积后的 36 个 H 元素和 6 个 g 分量均在 FP64 下归约, FP32 截断误差 ($\approx 10^{-7}$) 经 FP64 累加后被抑制。消融实验证实: CUDA 混合精度在全部 N 值上收敛率 ≥ 0.998 , 与全 FP64 的 FP64 自适应阻尼配置 (1.000) 无实质差异。

3.5 自适应阻尼策略

全 FP64 基线采用固定阻尼 $\lambda \equiv 2 \times 10^{-3}$ 。本文采用三阶段自适应策略 [3]:

迭代 0——距离初始化。 $e_{\text{pos}} > 0.5 \text{ m} \rightarrow \lambda = \lambda_{\text{far}} = 0.1$; $e_{\text{pos}} > 0.1 \text{ m} \rightarrow$ 线性插值; 否则 $\rightarrow \lambda = \lambda_{\text{base}} = 5 \times 10^{-4}$ 。

迭代 1+——Marquardt 误差驱动。 $r = e_{\text{pos}}^{(k)} / e_{\text{pos}}^{(k-1)}$: $r < 0.9 \rightarrow \lambda \times 0.7$ (趋 Gauss-Newton); $r > 1.1 \rightarrow \lambda \times 2.0$ (趋梯度下降); $0.9 \leq r \leq 1.1 \rightarrow \lambda$ 不变。

停滞超驰。 连续 12 次无改善 $\rightarrow \lambda \times 5.0$ 。 λ 全局钳位至 $[10^{-4}, 0.5]$ 。

表 3: 统一 Benchmark 标准配置

参数	配置	选择理由
机器人模型	UR10 (URDF, tool0)	代表性 6-DOF 工业机械臂 [15]
目标位姿	seed=42, $N = 100 \rightarrow 10,000$	可复现确定性生成
初始种子	zero_seed (全零关节角)	运动学最远端
主收敛阈值	10 mm / 5°	显著区分求解器质量
最大迭代次数	160	工业应用保守上限
重复次数	30 (含 3 次预热)	消除冷启动延迟
硬件平台	RTX 4060 Laptop, CUDA 12.6	Ada Lovelace, sm_89

4 实验设计

4.1 统一 Benchmark 配置

4.2 消融级别设计

注：描述性配置名称替代原内部消融代号，编译二进制名 (A0/A5/A7) 与 ABLATION_LEVEL 宏保留以便对照源码复现。消融逻辑按“内存层次 $\rightarrow$ 算法优化 $\rightarrow$ 精度优化”递进：A0 建立最简可运行基线；A1–A4 逐级叠加常量内存、PaddedMat6 $\times$ 8、寄存器 LDL^T 和 kernel 融合，验证每项内存层次优化的独立收益；A5 (FP64 自适应阻尼) 跃迁至算法级优化；A7 (CUDA 混合精度) 跃迁至精度优化。B1/B2/B4/B6 等中间级别仅作完整消融链记录，不参与主结论分析。

4.3 对比求解器与计时口径

对比求解器：cuRobo[6] (NVIDIA GPU IK 框架, PyTorch 后端, 配置 num_seeds=1, seed_solver_num_seeds=1, self_collision_check=False, use_cuda_graph=False。注：关闭 CUDA Graph 以保证与本文单 kernel 方案的公平对比)；PyRoki (JAX GPU IK, JIT 预热时间排除在计时外)；Orocos KDL[5] (C++ CPU 串行 IK, max 160 iter)；numeric_dls (Python/NumPy DLS 参考实现, max 100 iter, 固定 $\lambda = 10^{-3}$)。CPU 参考求解器使用旧阈值 (30 mm/30 $^\circ$) 和 home_seed 策略，仅作数量级参照 (GPU vs CPU 的粗粒度量级比较)，不参与 Medium 阈值下的 GPU 主性能排名。

CUDA 求解器使用 cudaEventElapsedTime 测量 kernel 设备端时间；cuRobo 使用 time.perf_counter() 测量全调用栈。两者计时工具不同，“加速比”为统一 benchmark 下的工程吞吐比值。

5 实验结果与分析

5.1 主对比实验

$N = 100$ 时 CUDA 混合精度每目标仅摊 8.9 μ s (0.89 ms/100)，cuRobo 的跨语言调用栈固定开销无法在小批量下有效摊销。即使在 FP64 全精度配置下，本文求解器在 $N = 100$ 时吞吐 (51,361 targets/s) 仍为 cuRobo 的 16.5 倍，验证了低 per-target 开销对小批量性能的决定性作用。

5.2 消融实验

自适应阻尼 (FP64 基线 $\rightarrow$ FP64 自适应阻尼) 是对收敛率影响最大的单一因素。基线采用固定阻尼 $\lambda \equiv 2 \times 10^{-3}$ ，在 Medium 阈值下收敛率崩塌至 52–83%—— $N = 500$ 时仅 52.2% 的目标在 160 次迭代内收敛， $N = 5000$ 时为 56.4%。原因在于 fixed- λ 无法根据目标距离自适应调节：远距离目标 ($e_{\text{pos}} > 0.5$ m) 的梯度方向在 λ 过大时收敛缓慢，近距离目标的 Hessian 近似在 λ 过小时趋向奇异。FP64 自适应阻尼通过距离初始化 + Marquardt 误差驱动 + 停滞超驰三阶段策略，在全部 N 值上将收敛率恢复至 100%。吞吐提升 (4–6 倍) 主要来自平均迭代次数的大幅下降： $N = 100$ 从 31.4 降至 12.9 (–59%)， $N = 500$ 从 79.7 降至 13.8 (–83%)， $N = 5000$ 从 73.0 降至 13.7 (–81%)。

混合精度 (FP64 自适应阻尼 $\rightarrow$ CUDA 混合精度) 在此基础上再提供 120–149% 吞吐提升，且收敛率保持 0.998+。该增益来源于 FK 和 Jacobian 计算中 FP64 $\rightarrow$ FP32 的精度切换——两项合计占每次迭代约 90% 的浮点运算量——在 Ada Lovelace 64:1 的 FP32:FP64 吞吐比下释放了显著的计算加速。需特别指出，内存层次优化 (B1 常量内存、B2 PaddedMat6 $\times$ 8) 在消融链中的独立贡献合计 < 5%，因为 UR10 工作集仅约 912 bytes，天然即可被每个 SM 的 128 KB L1 缓存高效容纳。这一发现与文献 [13] 的结论一致——

表 4: 关键消融级别定义

配置名称	编译目标	功能描述
FP64 基线	A0	全局内存, stride=6, 固定阻尼 $\lambda \equiv 2 \times 10^{-3}$
FP64 自适应阻尼	A5	+ 常量内存 +PaddedMat6 $\times$ 8+ 自适应阻尼
CUDA 混合精度	A7	+FP32 FK/Jacobian + FP64 H/g/LDL^T

表 5: CUDA 混合精度与 cuRobo 主对比 (Medium 10mm/5°)

N	CUDA-M/(t/s)	CUDA-M/ms	cuRobo/(t/s)	cuRobo/ms	比值	收敛率
100	112,414	0.89	3,118	32.1	36.1	1.000
500	158,251	3.16	15,844	31.6	10.0	0.998
1,000	148,412	6.74	31,611	31.6	4.7	0.998
5,000	168,683	29.64	155,059	32.3	1.09	0.9998

Bank 冲突消除对极小矩阵的收益有限。自适应阻尼和混合精度才是决定性能的两大主导因素。

5.3 Nsight Compute 剖析

计算吞吐率 (60–67%) 远超显存吞吐率 (1–2%), 判定 kernel 为计算密集型。Kernel 时间从 2,920 μ s 降至 827 μ s (–72%), 与吞吐提升 120–149% 相互印证。零局部内存溢出确认编译器寄存器分配的充分性。

5.4 全量程批量扩展性

在 $N = 100 \rightarrow 10,000$ 范围以 1,000 为步长进行全量程扫描 (12 个 N 值)。

本文求解器的线性扩展——与批量无关。 CUDA 混合精度在 12 个 N 值上的吞吐保持在 148k–174k targets/s 窄区间 ($\pm 8\%$, 以中位数 166k 为基准), GPU kernel 时间与 N 的线性拟合 $R^2 > 0.999$ 。这是 1 block-/target 映射的结构确定性结果: 每目标计算量完全确定 (FK + Jacobian + LDL^T + Update 在固定迭代轮数内完成), 无 block 间通信, 无动态内存分配, 无 GPU-CPU 同步, 无 sub-batch 划分策略——工作量线性增长 $\rightarrow$ 执行时间线性增长 $\rightarrow$ 吞吐恒定。批量无关性对实际工程应用具有直接价值: 用户无需对不同 N 值进行预扫描或参数调优, 吞吐可从事先的计算精确预测。

cuRobo 的批量振荡——非单调二元跳变。 相同 12 个 N 值上, cuRobo 呈现二元振荡模式: $N = 4,000/7,000/9,000/10,000$ 四个点触发约 230 ms 退化 (正常 32 ms 的 7 倍), 其余 N 值正常。退化具有非单调特征—— $N = 4,000$ 退化但 $N = 5,000$ 正常, $N = 7,000$ 退化但 $N = 8,000$ 正常, $N = 8,000$ 甚

至以 248,732 targets/s 创下 cuRobo 全量程最高吞吐。Nsight Systems CUDA API trace 揭示退化 N 值的 CUDA kernel 启动数为正常的 2.37 倍 (14,108 vs 5,945), cudaEventRecord/cudaStreamWaitEvent 调用数高达 13–14 倍。cudaMalloc/cudaFree 开销在两组 N 值下均 $< 1\%$ API 时间, 明确排除 PyTorch CUDA 缓存分配器作为退化主因, 指向 cuRobo 内部子批次划分策略的批量依赖性——特定 N 值触发了不同的 kernel launch 粒度, 而该粒度不由 N 的大小单调决定。本文求解器的单 kernel 全迭代封装在结构上对此类问题免疫: kernel launch 数恒为 1, 不存在子批次划分策略, 性能仅由 N 和目标位姿的收敛难度决定。

6 结论

本文针对机械臂批量 IK 中 6×6 小矩阵反复求解的性能瓶颈, 提出了四项 CUDA 底层加速设计: (1) 1 block/target 映射与单 kernel 全迭代封装; (2) 寄存器驻留的 6×6 LDL^T 求解器 (86 标量运算, 零寄存器溢出); (3) PaddedMat6 $\times$ 8 共享内存布局 (Bank 冲突 –63%); (4) FP32 FK/Jacobian + FP64 LDL^T 混合精度策略 (吞吐 +120–149%, 收敛率无退化)。在统一 UR10 benchmark 上, CUDA 混合精度在 $N = 100$ 时吞吐达 cuRobo 的 36.1 倍, 全量程 $N = 100 \rightarrow 10,000$ 上 GPU 时间严格线性 ($R^2 > 0.999$), 而 cuRobo 在 4/12 N 值上触发约 230 ms 退化模式——体现了底层单 kernel 方案相比框架型 GPU IK 实现在批量扩展确定性和性能可预测性上的结构性优势。未来工作将扩展至 7 自由度冗余机械臂的性能验证及多初始种子策略的 GPU 并行化。

表 6: 逐级消融实验结果 (Medium 10mm/5°)

N	FP64 基线/(t/s)	收敛率	FP64 阻尼/(t/s)	提升	CUDA-M/(t/s)	提升	收敛率
100	7,589	0.830	51,361	+577%	113,097	+120%	1.000
500	9,896	0.522	62,384	+530%	155,071	+149%	0.998
5,000	12,507	0.564	66,050	+428%	164,207	+149%	0.9998

表 7: Nsight Compute 剖析对比 ($N = 100$)

指标	FP64 全优化	CUDA-Mixed	变化
计算吞吐率/%	66.89	60.73	-9
显存吞吐率/%	1.56	1.16	-0.4
寄存器/线程	94	98	+4
占用率/%	32.51	33.30	≈ 0
Bank 冲突	3,522	1,295	-63%
L1 命中率/%	99.13	98.62	≈ 0
局部内存溢出	0	0	-
Kernel 时间/ μ s	2,920	827	-72%

参考文献

- [1] SICILIANO B, SCIAVICCO L, VILLANI L, et al. Robotics: modelling, planning and control [M]. 2nd ed. London: Springer, 2010.
- [2] BUSS S R. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods [J]. IEEE Journal of Robotics and Automation, 2004, 17(1): 1-19.
- [3] MARQUARDT D W. An algorithm for least-squares estimation of nonlinear parameters [J]. Journal of the Society for Industrial and Applied Mathematics, 1963, 11(2): 431-441.
- [4] 贾龙飞, 乔尚岭, 陶云飞, 等. 冗余机械臂逆运动学求解方法研究进展 [J]. 控制与决策, 2023, 38(12): 3267-3284.
- [5] BRUYNINCKX H. Open robot control software: the OROCOS project [C]//Proc. of the IEEE ICRA. Seoul: IEEE, 2001: 2523-2528.
- [6] SUNDARALINGAM B, HARIH S K S, FISHMAN A, et al. cuRobo: parallelized collision-free minimum-jerk robot motion generation [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2310.17274>.
- [7] YASUTAKE S, KINGSTON Z, PLANCHER B. HJCD-IK: GPU-accelerated inverse kinematics through batched hybrid Jacobian coordinate descent [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2510.07514>.
- [8] ABOELNASR M, et al. ManipulaPy: a GPU-accelerated Python framework for robotic manipulation [J]. Journal of Open Source Software, 2025, 10: 8490.
- [9] PERRAULT N, HO Q H, LAHIJANIAN M. Kino-PAX: highly parallel kinodynamic sampling-based planner [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2409.06807>.
- [10] CHEN L, IYER S R, KINGSTON Z. SPaSM: differentiable particle optimization for fast sequential manipulation [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2510.07674>.
- [11] ABDELFAH A, HAIDAR A, TOMOV S, et al. Batched one-sided factorizations of tiny matrices using GPUs: challenges and countermeasures [J]. Journal of Computational Science, 2018, 26: 226-236.
- [12] 刘世芳, 赵永华, 黄荣锋, 等. 基于批量 LU 分解的矩阵求逆在 GPU 上的有效实现 [J]. 软件学报, 2023, 34(11): 4952-4972.

- [13] SHRIDHAR S A. Optimized block-level matrix inversion kernels for small, batched matrices on GPUs [D]. 2024.
- [14] ABDELFAHATTAH A, HAIDAR A, TOMOV S, et al. Mixed-precision iterative refinement using tensor cores on GPUs to accelerate solution of linear systems [J]. Proceedings of the Royal Society A, 2021, 477(2253): 20200110.
- [15] Universal Robots. Universal Robots ROS2 description package (tag 4.3.1) [EB/OL]. [2025-06-14]. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [16] NVIDIA Corporation. CUDA C++ programming guide [EB/OL]. [2025-06-14]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [17] ABDELFAHATTAH A, TOMOV S, LUSZCZEK P, et al. GPU-based LU factorization and solve on batches of matrices with band structure [C]//Proc. of the Workshops of SC. Denver: ACM, 2023: 1670–1679.
- [18] ABUELSAMEN A, RANA S, et al. Industrial robot motion planning with GPUs: integration of cuRobo for extended DOF systems [EB/OL]. [2025-06-14]. <https://arxiv.org/abs/2508.04146>.

作者简介

刘小明 (19**-), 男, XXXX, 硕士, 主要研究方向为 GPU 并行计算、机器人运动学。

E-mail: 手机: 固话:

地址: 邮编: 身份证号码:

某某某 (19**-), 男/女, 教授, 博士, 主要研究方向为 XXXX。

E-mail: 手机: 固话:

地址: 邮编: 身份证号码: